

# Laporan Praktikum Minggu Ke-4 Kontrol Cerdas

## Minggu Ke-4

Nama : Mohammad Arief Rachman  
NIM : 224308060  
Kelas : TKA 7C  
Akun Github (Tautan) : <https://github.com/moharief0304>

### 1. Judul Laporan

*Reinforcement Learning for Autonomous Control*

### 2. Tujuan Percobaan

- Memahami konsep dasar *Reinforce Learning* (RL) dalam sistem kendali
- Mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
- Menggunakan *OpenAI Gym* sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk *version control* dan dokumentasi praktikum.

### 3. Landasan Teori

*Reinforcement Learning* (RL) adalah bagian dari *artificial intelligence* yang melatih algoritma dengan sistem *trial and error*. RL berinteraksi dengan lingkungannya dan mengamati konsekuensi atas tindakannya sebagai tanggapan atas penghargaan maupun hukuman yang diterima. Informasi yang dihasilkan dari setiap interaksi dengan lingkungan, digunakan RL untuk memperbarui pengetahuannya. *Reinforcement learning* dapat didefinisikan dengan belajar apa yang akan dilakukan pembelajaran dengan, pemetaan situasi dalam menentukan tindakan, dan memaksimalkan angka sinyal penghargaan yang bisa diperoleh dari lingkungannya. RL berinteraksi dengan lingkungannya dan mengamati konsekuensi atas tindakannya sehingga dapat belajar mengubah tingkah lakunya sendiri sebagai tanggapan atas penghargaan yang diterima. Informasi yang dihasilkan dari setiap interaksi dengan lingkungan yang kemudian digunakan RL untuk memperbarui pengetahuannya.

*Deep Q-learning* adalah algoritma pembelajaran penguatan yang menggabungkan *Q-learning* dengan teknik pembelajaran mendalam untuk mempelajari kebijakan optimal dalam pengambilan keputusan di lingkungan yang kompleks. DQN, atau *Deep Q-Network*, adalah implementasi spesifik dari *deep Q-learning* yang menggunakan jaringan saraf tiruan dalam untuk mengaproksimasi fungsi aksi-nilai. Perbedaan utama antara keduanya adalah *deep Q-learning* merupakan konsep umum, sementara DQN merupakan arsitektur dan algoritma spesifik untuk mengimplementasikan *deep Q-learning*.

*LunarLander-v2* adalah lingkungan pembelajaran penguatan klasik yang disediakan oleh pustaka Gym OpenAI. Tujuannya adalah mengembangkan agen cerdas yang mampu mendaratkan modul lunar dengan aman di permukaan bulan dengan konsumsi bahan bakar minimal. Artikel ini akan membahas penyelesaian masalah yang menantang ini menggunakan Deep Q-Networks (DQN), sebuah teknik pembelajaran penguatan mendalam yang populer.

#### 4. Analisis dan Diskusi

Pada praktikum minggu keempat ini, kita melakukan eksperimen dengan menerapkan algoritma *Deep Q-Network* (DQN) untuk memecahkan masalah lingkungan *MountainCar-v0* menggunakan pendekatan reinforcement learning. Tujuan utamanya adalah melatih agen supaya bisa sampai ke puncak gunung dengan cara memanfaatkan pengalaman-pengalaman yang sudah dikumpulkan selama proses pembelajaran.

Program ini dimulai dengan membuat kelas *DQNAgent* yang bertugas mengatur semua logika pembelajaran. Yang menarik dari agen ini adalah dia punya memory buffer berbentuk deque yang berguna untuk menyimpan berbagai pengalaman (*experience replay*). Jadi pembelajaran tidak cuma bergantung pada pengalaman yang baru saja terjadi, tapi juga bisa belajar dari pengalaman-pengalaman lama yang sudah tersimpan.

Cara kerja agen dalam mengambil keputusan itu menggunakan konsep eksplorasi dan eksploitasi yang dikontrol lewat parameter epsilon. Di awal-awal, agen lebih suka pilih aksi secara random (eksplorasi) karena dia belum tau apa-apa. Tapi seiring berjalannya waktu, nilai epsilon ini akan turun pelan-pelan (**epsilon\_decay**), makanya agen mulai lebih percaya sama pengalaman yang udah dia kumpulkan dan lebih sering pilih aksi berdasarkan pembelajaran sebelumnya (eksploitasi).

Untuk bagian utama programnya, lingkungan **MountainCar-v0** dari *Gymnasium* diatur dengan mode human render supaya kita bisa lihat visualisasi pergerakan mobilnya secara langsung. Proses pelatihan dilakukan dalam beberapa episode yang sudah ditentukan. Di setiap episode, agen akan mengamati kondisi lingkungan (**state**), memutuskan aksi apa yang mau diambil (**act()**), nerima feedback berupa reward, dan menyimpan semua pengalaman itu ke memory buffer. Sistem reward yang diterapkan cukup sederhana tapi efektif. Kalau agen berhasil mencapai tujuan (**terminated**), dia dapat reward yang lebih besar. Tapi kalau masih dalam proses perjalanan, dia dikasih penalti kecil supaya termotivasi untuk terus eksplorasi dan cari jalan yang lebih baik. Ada juga integrasi dengan Pygame untuk deteksi kalau user nutup jendela game. Kalau jendela ditutup, program langsung berhenti dan environment ditutup pakai **env.close()**.

Secara keseluruhan, program ini menunjukkan gimana agen bisa belajar menggunakan DQN, dimana dia belajar dari pengalaman untuk mengoptimalkan cara bergerak dalam lingkungan **MountainCar-v0**.

Kalau dilihat dari sisi analisis, program DQN ini cukup bagus untuk melatih agen menyelesaikan masalah MountainCar-v0 karena menggunakan *experience replay* yang bikin pembelajaran jadi lebih stabil. Strategi eksplorasi dan eksploitasinya juga udah tepat dengan nilai epsilon yang turun secara bertahap, jadi agen bisa nemuin strategi terbaik tanpa langsung stuck di pola tertentu.

Sistem reward yang dirancang juga udah cukup baik untuk motivasi agen supaya mau capai tujuan, meskipun mungkin bisa ditingkatkan lagi dengan kasih reward tambahan pas agen mulai mendekati puncak. Penggunaan sampel random dalam pelatihan membantu menjaga stabilitas pembelajaran, tapi tetap ada potensi risiko overfitting yang bisa diatasi kalau kita implementasikan metode *prioritized experience replay* di masa depan.

## 5. Assignment

Dalam praktikum kali ini, kami membuat program yang mengimplementasikan algoritma *Deep Q-Network* (DQN) untuk mengatasi permasalahan **MountainCar-v0** dengan menggunakan *library Gymnasium*. Tujuannya adalah melatih agen supaya bisa naik sampai ke puncak bukit dengan cara belajar dari pengalaman-pengalaman yang sudah dialami sebelumnya.

Yang menarik dari program ini adalah penerapan konsep *experience replay*, dimana agen bisa menyimpan pengalaman-pengalamannya di memory buffer dan menggunakannya lagi saat proses training. Ini sangat membantu untuk menjaga kestabilan pembelajaran dan mencegah agen terlalu fokus sama pengalaman yang baru-baru ini aja.

Program yang kami buat punya beberapa fungsi utama yang mendukung proses pelatihan agen. Di dalam kelas *DQNAgent*, ada fungsi **act()** yang berperan dalam menentukan aksi agen berdasarkan strategi **epsilon-greedy**. Dengan begitu, agen masih punya kesempatan untuk menjelajahi lingkungan dan tidak langsung stuck di strategi tertentu. Kemudian ada fungsi **remember()** yang tugasnya menyimpan pengalaman yang didapat selama training, dan fungsi **replay()** yang bertugas mengambil sample pengalaman secara random dari memory buffer dan mengupdate nilai Q dengan cara melatih ulang modelnya menggunakan data tersebut.

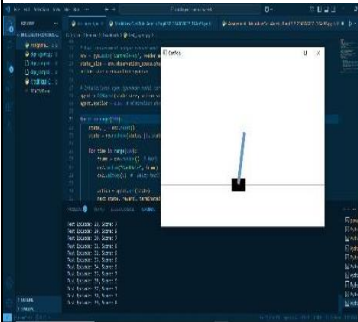
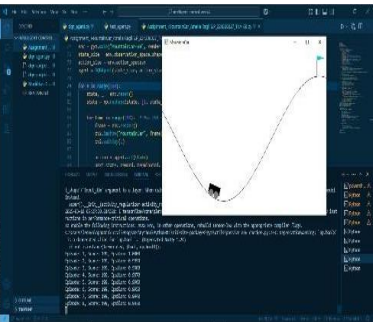
Proses setiap episode dimulai dengan mereset lingkungan, kemudian agen akan terus-menerus memilih aksi, menerima reward, menyimpan pengalaman, dan mengupdate model berdasarkan data yang terkumpul. Kalau jumlah pengalaman di *memory buffer* sudah cukup, agen akan menjalankan proses training untuk memperbaiki cara pengambilan keputusannya.

Selain itu, program ini juga terintegrasi dengan Pygame untuk menangani input dari user. Jadi kalau jendela gamenya ditutup sama user, program akan otomatis berhenti. Tapi karena program menggunakan mode *human render*, visualisasi ini bisa bikin proses training jadi lebih lambat. Makanya mungkin bisa dipertimbangkan untuk matikan tampilan selama training dan baru dinyalakan pas evaluasi aja.

Ada beberapa parameter yang bisa kita modifikasi untuk eksperimen, seperti gamma, epsilon decay, atau arsitektur neural networknya untuk lihat gimana perubahan tersebut bisa mempengaruhi performa agen. Selain itu, metode seperti prioritized experience replay juga bisa ditambahkan untuk meningkatkan efisiensi pembelajaran.

## 6. Data dan Output Hasil Pengamatan

| No. | Variabel                    | Hasil Pengamatan                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |                             | CartPole-v1                                                                                                                                                                                                                                                                                                                                                                                        | MountainCar-v0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 1.  | Waktu Pelatihan             | <p>Lebih cepat mencapai performa optimal dalam beberapa ratus Episode.</p> <pre> _name__ == '__main__': env = gym.make('CartPole-v1') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 64 target_update_interval = 10 # Interval to u </pre>                                                    | <p>Membutuhkan lebih banyak waktu untuk mencapai keberhasilan yang stabil.</p> <pre> _name__ == '__main__': env = gym.make('MountainCar-v0') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 64 target_update_interval = 10 # Interval to u </pre>                                                                                                                                          |
| 2.  | Kondisi Awal untuk Scorenya | <p>Skor bervariasi tergantung seberapa lama tiang tetap seimbang.</p> <pre> Test Episode: 3, Score: 83 Test Episode: 4, Score: 48 Test Episode: 5, Score: 55 Test Episode: 6, Score: 452 Test Episode: 7, Score: 37 Test Episode: 8, Score: 80 Test Episode: 9, Score: 35 Test Episode: 10, Score: 105 Test Episode: 11, Score: 20 Test Episode: 12, Score: 102 Test Episode: 13, Score: 39 </pre> | <p>Skor awal mencapai 199 karena maksimal langkah dalam satu episode adalah 200.</p> <pre> Test Episode: 2, Score: 199, Test Episode: 3, Score: 199, Test Episode: 4, Score: 199, Test Episode: 5, Score: 199, Test Episode: 6, Score: 199, Test Episode: 7, Score: 199, Test Episode: 8, Score: 199, Test Episode: 9, Score: 199, Test Episode: 10, Score: 199, Test Episode: 11, Score: 199, Test Episode: 12, Score: 199, Test Episode: 13, Score: 199, Test Episode: 14, Score: 199, </pre> |
| 3.  | Kondisi Akhir               | <p>Episode berakhir jika tiang jatuh atau batas langkah maksimum Tercapai.</p>                                                                                                                                                                                                                                                                                                                     | <p>Episode berakhir jika mobil mencapai puncak bukit (bendera).</p>                                                                                                                                                                                                                                                                                                                                                                                                                             |

|    |               |                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                       |
|----|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4. | Prinsip Kerja | <p>Sistem ini mengikuti hukum fisika, di mana jika tongkat mulai miring, gerobak harus segera bergerak ke arah kemiringan untuk menyeimbangkannya.</p> <p>Jadi, tongkat harus tetap seimbang di atas gerobak yang bergerak ke kiri atau kanan.</p> | <p>Sistem ini menampilkan sebuah mobil kecil berada di lembah di antara dua bukit dan harus mencapai puncak salah satu bukit.</p> <p>Mobil tidak memiliki tenaga yang cukup untuk langsung naik, sehingga harus mengandalkan momentum dengan cara mengayun bolak-balik agar dapat mencapai ketinggian yang cukup.</p> |
| 5. | Hasil         | <p>Agen berhasil menjaga tiang tetap tegak dalam simulasi CartPole.</p>                                                                                         | <p>Agen mampu menggerakkan mobil naik-turun bukit hingga akhirnya mencapai bendera (goal).</p>                                                                                                                                    |

## 7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan dapat disimpulkan bahwa:

- Metode prioritized experience replay bisa dipakai untuk membuat agen lebih cepat belajar dari pengalaman-pengalaman yang punya dampak lebih besar terhadap keberhasilannya
- Mode human render dalam simulasi ternyata membuat proses training jadi lambat banget. Makanya, waktu training sebaiknya visual display-nya dimatikan dulu dan cuma dinyalakan pas mau evaluasi hasilnya saja
- Lingkungan yang reward-nya jarang kayak MountainCar ini butuh waktu eksplorasi yang lebih panjang dan replay buffer yang lebih besar supaya agen dapat mengumpulkan pengalaman yang cukup untuk nemuin strategi yang efektif.
- Penggunaan target network ternyata sangat membantu untuk menjaga kestabilan pembelajaran, jadi perubahan nilai Q tidak terlalu ekstrem dan mendadak
- Dengan menggunakan DQN, agen bisa belajar bahwa hanya jalan maju terus itu tidak cukup untuk sampai ke tujuan. Agen mulai paham kalau terkadang dia harus mundur dulu untuk ngumpulin momentum sebelum akhirnya bisa naik sampai ke puncak.

## 8. Saran

Untuk meningkatkan efektivitas pembelajaran agen dalam menyelesaikan permasalahan *MountainCar-v0*, ada beberapa hal yang dapat dioptimalkan. Salah satunya adalah penyesuaian parameter seperti *gamma*, *epsilon decay*, dan ukuran *memory buffer* agar agen dapat belajar lebih efisien dan menemukan strategi optimal dengan lebih cepat. Dari segi efisiensi, karena *MountainCar-v0* membutuhkan banyak episode untuk belajar, sebaiknya tampilan visual (*human render*) dinonaktifkan selama pelatihan agar tidak memperlambat proses. Visualisasi dapat diaktifkan kembali saat evaluasi untuk melihat hasil akhir dari pembelajaran agen. Dengan berbagai perbaikan ini, diharapkan agen dapat belajar lebih cepat, lebih stabil, dan lebih efisien dalam menyelesaikan tantangan *MountainCar-v0*.



## 9. Daftar Pustaka

K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 5, 2017.

*What is Deep Q-Networks (DQN)? | Activeloop*

*Glossary.* [www.activeloop.ai/resources/glossary/deep-q-networks-dqn](http://www.activeloop.ai/resources/glossary/deep-q-networks-dqn).

Universitas Airlangga. "Reinforcement Learning, Mesin Pembelajaran Yang Belajar Dari Kesalahan." *Universitas Airlangga Official Website*, 15 May 2023, [unair.ac.id/reinforcement-learning-mesin-pembelajaran-yang-belajar-dari-kesalahan](http://unair.ac.id/reinforcement-learning-mesin-pembelajaran-yang-belajar-dari-kesalahan)

R. Giryes and M. Elad, "Reinforcement Learning: A Survey," *Eur. Signal Process. Conf.*, pp. 1–2, 2011.